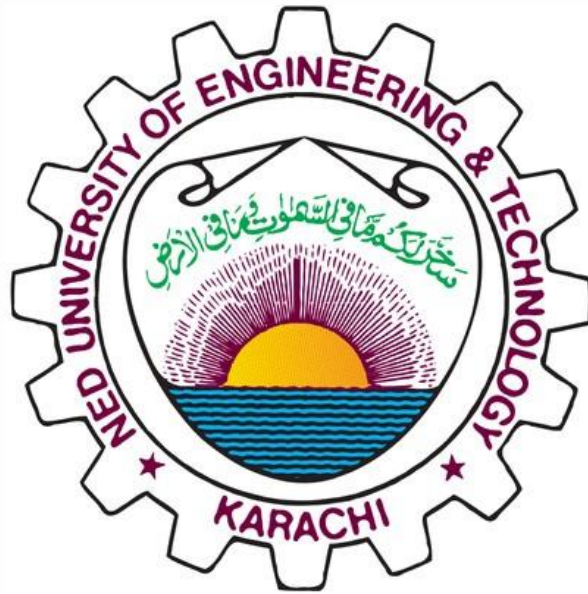




PROJECT REPORT

SPACE INVADERS GAME

Project Report

Project Name: Space Invaders Game

Course Name: Object-Oriented Programming

Course Code: CT-260

Project Contributors:

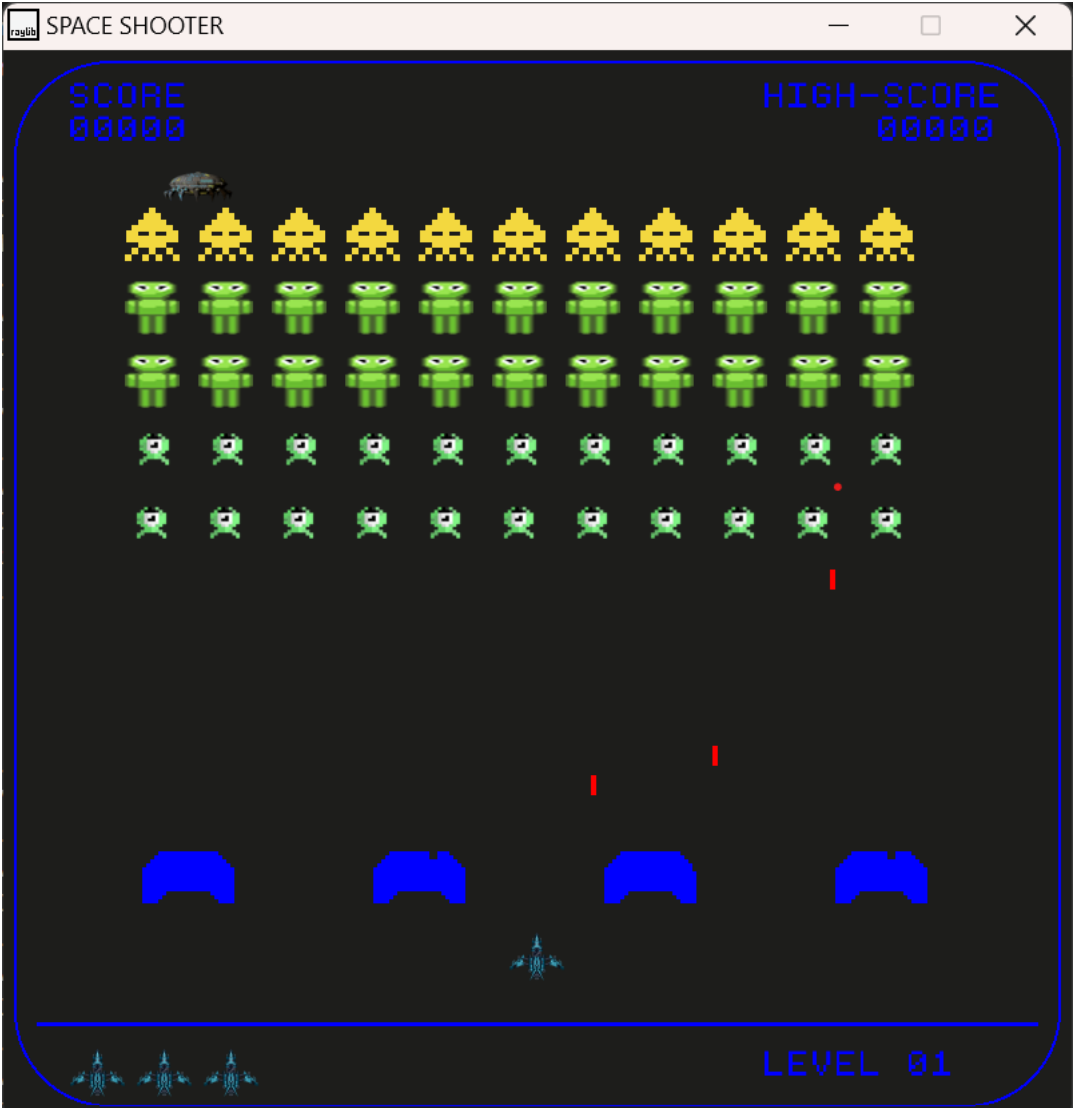
SYED MUHAMMAD AHMED	(CT-23049)
---------------------	------------

MUHAMMAD SAMEED	(CT-23046)
-----------------	------------

MUREED HUSSAIN	(CT-23036)
----------------	------------

FAIQ AHMED	(CT-23030)
------------	------------

USER INTERFACE



PROJECT DETAILS

FEATURES OF THE PROJECT:

1. Graphics and Animation:

Custom Font and Textures: Utilizes a custom font (monogram.ttf) for text rendering and custom textures for the spaceship and other game elements.

Smooth Animations: Smooth movement and animations for the spaceship, lasers, and aliens.

Screen Resolution Handling: The game window size and element positions are calculated based on screen resolution, ensuring proper display on various devices.

2. Sound and Music:

Background Music: Continuous background music adds to the immersive experience.

Sound Effects: Sound effects for laser firing and explosions enhance the gameplay.

3. Game Mechanics:

Player Controls: Responsive controls for moving the spaceship left, right, and firing lasers.

Laser Shooting: Both the player and aliens can shoot lasers, adding a dynamic element to the game.

Alien Movement: Aliens move horizontally and change direction upon reaching the screen edges, occasionally moving down.

Collision Detection: Comprehensive collision detection between lasers, aliens, obstacles, and the spaceship.

4. Game States:

Run and Game Over States: The game handles running and game over states, allowing the player to restart the game upon pressing Enter.

Level and Score Display: The current level, score, and high score are displayed on the screen.

Lives System: The player has a limited number of lives, with the game ending when lives are depleted.

5. File I/O:

High Score Saving and Loading: High scores are saved to and loaded from a file, allowing persistent storage of the player's achievements.

6. Game Elements and Enemies:

Spaceship: Player-controlled spaceship with smooth movement and laser shooting capabilities.

Aliens: Multiple types of aliens with different point values, creating varied challenges.

Obstacles: Destructible obstacles that block lasers and add strategic depth.

Mystery Ship: A special ship that appears occasionally, providing bonus points when destroyed.

7. Game Loop and Timing:

Main Game Loop: The game runs in a continuous loop, updating game states and rendering graphics.

Frame Rate Control: The game is set to run at 60 frames per second for a consistent experience.

Timed Events: Events such as alien laser shooting and mystery ship spawning are controlled by timers.

8. User Interface:

Score Display: Current score and high score are displayed with formatted text.

Lives Display: Remaining lives are visually represented with spaceship icons.

Game Over Message: A clear game over message is displayed when the player loses all lives.

OOP UTILIZATION:

1. Classes and Objects:

Spaceship Class: Represents the player's spaceship. It encapsulates attributes like position, image, and methods for moving, drawing, firing lasers, and resetting the spaceship.

Laser Class: Represents a laser shot by the spaceship or aliens. It includes attributes like position, speed, and methods for updating its position, drawing itself, and getting its rectangle for collision detection.

Game Class: Manages the overall game logic, including updating the game state, handling input, drawing game elements, and managing the game loop.

Other Classes: Obstacle, Alien, and MysteryShip.

2. Encapsulation:

Each class hides its internal implementation details and exposes a public interface through methods.

For instance:

Spaceship Class:

Private attributes: image, position, lastfiretime, lasersound

Public methods: moveleft(), moveright(), firelaser(), draw(), getrec(), reset()

3. Inheritance and Polymorphism:

It's possible that the alien, obstacle, and ship classes could be designed to inherit from a common base class if they share common attributes or methods.

4. Abstraction:

Complex implementation details are abstracted away behind simple method calls.

For example:

The **firelaser()** method in the spaceship class abstracts the details of creating a new laser, setting its initial position, and playing the laser sound.

The **update()** method in the game class abstracts the complex logic of updating various game elements like the spaceship, aliens, and lasers.

5. Composition:

The game class is composed of multiple objects from other classes:

i.e;

spaceship s: The player's spaceship.

std::vector<laser> alienlaser: A collection of alien lasers.

std::vector<obstacle> obstacles: A collection of obstacles.

WHAT NEW WE LEARNED?

CONCEPTS:

Game Loop: Understanding the continuous cycle of processing input, updating the game state, and rendering the game. The game loop is the core of any game engine.

Collision Detection: Implementing algorithms to detect collisions between game objects (e.g., lasers, spaceships, aliens).

Entity Management: Managing different game entities (spaceship, aliens, obstacles) using classes and objects.

Event Handling: Handling user inputs and game events, such as key presses and timed events.

State Management: Managing different game states (e.g., running, game over) and transitioning between them.

File I/O: Reading and writing data to files, such as saving and loading high scores.

Timers and Intervals: Using timers to control events, like alien laser firing and mystery ship spawning, at regular intervals.

Randomization: Implementing randomness for game elements like alien movements and mystery ship spawns.

Resource Management: Loading, using, and unloading resources (textures, sounds) efficiently.

Library:

Raylib: The primary library used for game development. It simplifies graphics rendering, input handling, and audio management.

raylib.h: The core header file for Raylib functions.

LoadTexture: Loading images as textures.

DrawTextureV: Drawing textures at specific screen coordinates.

InitWindow: Initializing the game window.

SetTargetFPS: Setting the game's frame rate.

BeginDrawing/EndDrawing: Wrapping the rendering code.

IsKeyDown: Checking if a key is currently pressed.

PlaySound: Playing sound effects.

LoadMusicStream/PlayMusicStream: Loading and playing background music.

CheckCollisionRecs: Checking collision between two rectangles.

UnloadTexture/UnloadSound: Unloading resources.

SOURCE CODE

CLASS LASER:

UML DIAGRAM:

Laser	
-	position : Vector2
-	speed : int
+	active : bool
+	laser(position : Vector2, speed : int)
+	update() : void
+	draw() : void
+	getrec : Rectangle

CLASS CODE:

```
#pragma once
#include<raylib.h>

class laser{
    private:
        Vector2 position;
        int speed;

    public:
        laser(Vector2 position,int speed);
        void update();
        void draw();
        Rectangle getrec();
        bool active;
};
```

IMPLEMENTATION:

```
#include "laser.hpp"
#include <iostream>

laser::laser(Vector2 position, int speed) {
    this->position = position;
    this->speed = speed;
    active = true;
```

```
}

void laser::draw() {
    if (active) {
        DrawRectangle(position.x, position.y, 4, 15, {255, 0, 0, 255});
    }
}

void laser::update() {
    position.y += speed;
    if (active) {
        if (position.y > GetScreenHeight() - 100 || position.y < 25) {
            active = false;
        }
    }
}

Rectangle laser::getrec() {
    Rectangle rec;
    rec.x = position.x;
    rec.y = position.y;
    rec.width = 4;
    rec.height = 15;
    return rec;
}
```

CLASS SPACESHIP:

UML DIAGRAM:

Spaceship
- image : Texture2D
- position : Vector2
- lastfiretime : double
- lasersound : Sound
+ lasers : std::vector<laser>
+ spaceship()
+ ~spaceship()
+ draw() : void
+ moveleft() : void
+ moveright() : void
+ firelaser() : void
+ getrec() : Rectangle
+ reset() : void

CLASS CODE:

```
#pragma once
#include<raylib.h>
#include"laser.hpp"
#include<vector>

class spaceship{
private:
    Texture2D image;
    Vector2 position;
    double lastfiretime;
    Sound lasersound;

public:
    std::vector<laser> lasers;
    spaceship();
    ~spaceship();
    void moveleft();
    void moveright();
    void draw();
    void firelaser();
    Rectangle getrec();
    void reset();
};
```

IMPLEMENTATION:

```
#include "spaceship.hpp"

spaceship::spaceship(){
    image=LoadTexture("graphics/a.png");
    position.x=(GetScreenWidth()-image.width)/2;
    position.y=GetScreenHeight()-image.height-100;
    lastfiretime=0.0;
    lasersound=LoadSound("sounds/laser.mp3");
}

spaceship::~spaceship() {
    UnloadTexture(image);
    UnloadSound(lasersound);
}

void spaceship::draw() {
    DrawTextureV(image,position,WHITE);
}

void spaceship::moveleft() {
    position.x-=7;
    if(position.x<25) {
        position.x=25;
    }
}

void spaceship::moveright() {
    position.x+=7;
    if(position.x>GetScreenWidth()-image.width-25){
        position.x=GetScreenWidth()-image.width-25;
    }
}

void spaceship::firelaser() {
    if(GetTime()-lastfiretime>=0.35){
        lasers.push_back(laser({position.x+image.width/2-2,position.y},-6));
        lastfiretime=GetTime();
        PlaySound(lasersound);
    }
}

Rectangle spaceship::getrec() {
    return {position.x,position.y,float(image.width),float(image.height)};
}
```

```
void spaceship::reset() {
    position.x=(GetScreenWidth()-image.width)/2.0f;
    position.y=GetScreenHeight()-image.height-100;
    lasers.clear();
}
```

CLASS ALIEN:

UML DIAGRAM:

Alien
+ type : int
+ position : Vector2
+ alienimages[3] : Texture2D (static)
+ alien(type : int, position : Vector2)
+ update(direcrtion : int) : void
+ draw() : void
+ gettype() : int
+ unloadimages() : void (static)
+ getrect() : Rectangle

CLASS CODE:

```
#pragma once
#include<raylib.h>

class alien{
public:
    int type;
    Vector2 position;
    static Texture2D alienimages[3];
    alien(int type,Vector2 position);
    void update(int direction);
    void draw();
    int gettype();
    static void unloadimages();
    Rectangle getrect();
};
```

IMPLEMENTATION:

```
#include "alien.hpp"
```

```

Texture2D alien::alienimages[3]={};

alien::alien(int type, Vector2 position) {
    this->type=type;
    this->position=position;
    if(alienimages[type-1].id==0){
        switch(type){
            case 1:
                alienimages[0]=LoadTexture("graphics/jadu.png");
                break;
            case 2:
                alienimages[1]=LoadTexture("graphics/jadu2.png");
                break;
            case 3:
                alienimages[2]=LoadTexture("graphics/alien_3.png");
                break;
            default:
                alienimages[0]=LoadTexture("graphics/jadu.png");
                break;
        }
    }
}

void alien::draw() {
    DrawTextureV(alienimages[type-1],position,WHITE);
}

int alien::gettype() {
    return type;
}

void alien::unloadimages() {
    for(int i=0;i<4;i++){
        UnloadTexture(alienimages[i]);
    }
}

Rectangle alien::getrect() {
    return {position.x,position.y,float(alienimages[type-1].width),float(alienimages[type-1].height)};
}

void alien::update(int direction) {
    position.x+=direction;
}

```

CLASS MYSTERYSHIP:

UML DIAGRAM:

MysteryShip
- position : Vector2
- image : Texture2D
- speed : int
+ alive : bool
+ mysteryship()
+ ~mysteryship()
+ update() : void
+ draw() : void
+ spawn : void
+ getrec() : Rectangle

CLASS CODE:

```
#pragma once
#include<raylib.h>

class mysteryship{
    private:
        Vector2 position;
        Texture2D image;
        int speed;

    public:
        bool alive;
        mysteryship();
        ~mysteryship();
        void update();
        void draw();
        void spawn();
        Rectangle getrec();
};
```

IMPLEMENTATION:

```
#include "mysteryship.hpp"

mysteryship::mysteryship() {
    image=LoadTexture("graphics/alienspaceship.png");
```

```

        alive=false;
    }

    mysteryship::~mysteryship() {
        UnloadTexture(image);
    }

    void mysteryship::update() {
        if(alive){
            position.x+=speed;
            if(position.x>GetScreenWidth()-image.width-25||position.x<25){
                alive=false;
            }
        }
    }

    void mysteryship::draw() {
        if(alive){
            DrawTextureV(image,position,WHITE);
        }
    }

    void mysteryship::spawn() {
        position.y=90;
        int side=GetRandomValue(0,1);
        if(side==0){
            position.x=25;
            speed=3;
        } else {
            position.x=GetScreenWidth()-image.width-25;
            speed=-3;
        }
        alive=true;
    }

    Rectangle mysteryship::getrec() {
        if(alive){
            return{position.x,position.y,float(image.width),float(image.height)};
        } else {
            return{position.x,position.y,0,0};
        }
    }
}

```

CLASS BLOCK:

UML DIAGRAM:

BLOCK	
-	position : Vector2
+	Block(position : Vector2)
+	draw() : void
+	getrec() : Rectangle

CLASS CODE:

```
# pragma once
#include<raylib.h>

class Block{
    private:
        Vector2 position;

    public:
        Block(Vector2 position);
        void draw();
        Rectangle getrec();
};
```

IMPLEMENTATION

```
#include "block.hpp"

Block::Block(Vector2 position) {
    this->position=position;
}

void Block::draw() {
    DrawRectangle(position.x,position.y,3,3,{0,0,255,255});
}

Rectangle Block::getrec() {
    Rectangle rec;
    rec.x=position.x;
    rec.y=position.y;
    rec.width=3;
    rec.height=3;
    return rec;
}
```

CLASS OBSTACLE:

UML DIAGRAM:

Obstacle
+ position : Vector2
+ block : std::vector<Block>
+ grid : std::vector<std::vector<int>> (static)
+ obstacle(position : Vector2)
+ draw() : void

CLASS CODE:

```
#pragma once
#include<vector>
#include"block.hpp"

class obstacle {
public:
    Vector2 position;
    std::vector<Block> blocks;
    static std::vector<std::vector<int>> grid;
    obstacle(Vector2 position);
    void draw();
};
```

IMPLEMENTATION:

```
#include "obstacle.hpp"

std::vector<std::vector<int>>>obstacle:: grid = {
    {0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0},
    {0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0},
    {0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1},
}
```

```

        {1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1}
    };

    obstacle::obstacle(Vector2 position) {
        this->position = position;

        for(unsigned int row=0;row<grid.size();++row){
            for(unsigned int column=0;column<grid[0].size();column++){
                if(grid[row][column]==1){
                    float position_x=position.x+column*3;
                    float position_y=position.y+row*3;
                    Block block=Block({position_x,position_y});
                    blocks.push_back(block);
                }
            }
        }
    }

    void obstacle::draw() {
        for(auto& block: blocks){
            block.draw();
        }
    }
}

```

CLASS GAME:

UML DIAGRAM:

Game
- s: spaceship
- m: mysteryship
- obstacles: std::vector<obstacle>
- aliens: std::vector<alien>
- aliensdirection: int
- alienlasers: std::vector<laser>
- alienlasershootinterval: constexpr static float
- lasttimealienfired: float
- mysteryshipspawninterval: float
- timelastspawn: float
- explotionsound: Sound
+ run: bool
+ lives: int
+ score: int
+ highscore: int
+ music: Music
+ game()
+ ~game()
+ draw(): void

+ update(): void
+ handleinput(): void
- initgame(): void
- reset(): void
- deleteinactivelasers(): void
- createobstacles(): std::vector<obstacle>
- createaliens(): std::vector<alien>
- movealiens(): void
- movedownaliens(distance: int): void
- alienshootlaser(): void
- checkforcollisions(): void
- GameOver(): void
- checkforhighscore(): void
- savehighscoretofile(highscore: int): void
- loadhighscorefromfile(): int

CLASS CODE:

```
#pragma once
#include<raylib.h>
#include"spaceship.hpp"
#include"obstacle.hpp"
#include"alien.hpp"
#include"mysteryship.hpp"

class game{
private:
    spaceship s;
    mysteryship m;
    std::vector<obstacle> obstacles;
    std::vector<alien> aliens;
    std::vector<laser> alienlaser;
    int aliensdirection;
    constexpr static float alienlasershootinterval = 0.35;
    float lasttimealienfired;
    float mysteryshipspawninterval;
    float timelastspawn;
    Sound explotionsound;
    void initgame();
    void reset();
    std::vector<obstacle> createobstacles();
    std::vector<alien> createaliens();
    void deleteinactivelaser();
    void movealiens();
    void movedownaliens(int distance);
    void alienshootlaser();
    void checkforcollision();
```

```

int loadhighscorefromfile();
void savehighscoretofile(int highscore);
void checkforhighscore();
void GameOver();

public:
bool run;
int lives;
int score;
int highscore;
Music music;
game();
~game();
void draw();
void update();
void handleinput();
};

```

IMPLEMENTATION:

```

#include "game.hpp"
#include<iostream>
#include<fstream>

game::game() {
    music=LoadMusicStream("sounds/background.mp3");
    PlayMusicStream(music);
    explotionsound=LoadSound("sounds/explosion.mp3");

    initgame();
}

game::~game() {
    alien::unloadimages();
    UnloadMusicStream(music);
    UnloadSound(explotionsound);
}

void::game::update() {
    if(run){
        double currenttime=GetTime();
        if(currenttime-timelastspawn>mysteryshipspawninterval){
            m.spawn();
            timelastspawn=GetTime();
            mysteryshipspawninterval=GetRandomValue(10,20);
        }
    }
}

```

```

        for(auto& laser:s.lasers){
            laser.update();
        }
        movealiens();
        alienshootlaser();
        for(auto& laser:alienlaser){
            laser.update();
        }
        deleteinactivelaser();
        m.update();
        checkforcollision();
    }
    else{
        if(IsKeyDown(KEY_ENTER)){
            reset();
            initgame();
        }
    }
}

void game::draw() {
    s.draw();
    for(auto& laser:s.lasers){
        laser.draw();
    }
    for(auto& obstacle:obstacles){
        obstacle.draw();
    }
    for(auto& alien:aliens){
        alien.draw();
    }
    for(auto& laser:alienlaser){
        laser.draw();
    }
    m.draw();
}

void game::handleinput() {
    if(run){
        if(IsKeyDown(KEY_LEFT)){
            s.moveleft();
        }
        else if(IsKeyDown(KEY_RIGHT)){
            s.moveright();
        }
        else if(IsKeyDown(KEY_SPACE)){
            s.firelaser();
        }
    }
}

```

```

    }
}

void game::initgame() {
    obstacles=createobstacles();
    aliens=createaliens();
    aliensdirection=1;
    lasttimealienfired=0.0;
    m.spawn();
    timelastspawn=0.0;
    mysteryshipspawninterval=GetRandomValue(10,20);
    lives=3;
    score=0;
    highscore=loadhighscorefromfile();
    run=true;
}

void game::reset() {
    s.reset();
    aliens.clear();
    alienlaser.clear();
    obstacles.clear();
}

void game::deleteinactivelaser() {
    for(auto it=s.lasers.begin();it!=s.lasers.end();){
        if(!it->active){
            it=s.lasers.erase(it);
        }
        else{
            ++it;
        }
    }
    for(auto it=alienlaser.begin();it!=alienlaser.end();){
        if(!it->active){
            it=alienlaser.erase(it);
        }
        else{
            ++it;
        }
    }
}

void game::movedownaliens(int distance) {
    for(auto& alien:aliens){
        alien.position.y+=distance;
    }
}

```

```

void game::checkforcollision() {
    //spaceship laser
    for(auto& laser:s.lasers){
        auto it=aliens.begin();
        while(it!=aliens.end()){
            if(CheckCollisionRecs(it->getrect(),laser.getrec())){
                PlaySound(explotionsound);
                if(it->type==1){
                    score+=100;
                }
                else if(it->type==2){
                    score+=200;
                }
                else if(it->type==3){
                    score+=300;
                }
                checkforhighscore();
                it=aliens.erase(it);
                laser.active=false;
            }
            else{
                ++it;
            }
        };
        for(auto& obstacle:obstacles){
            auto it=obstacle.blocks.begin();
            while(it!=obstacle.blocks.end()){
                if(CheckCollisionRecs(it->getrec(),laser.getrec())){
                    it=obstacle.blocks.erase(it);
                    laser.active=false;
                }
                else{
                    ++it;
                }
            }
        }
        if(CheckCollisionRecs(m.getrec(),laser.getrec())){
            m.alive=false;
            laser.active=false;
            score+=500;
            checkforhighscore();
            PlaySound(explotionsound);
        }
    }
    //Alien lasers
    for(auto& laser:alienlaser){
        if(CheckCollisionRecs(laser.getrec(),s.getrec())){
            laser.active=false;
            lives--;
        }
    }
}

```

```

        if(lives==0){
            GameOver();
        }
    }
    for(auto& obstacle:obstacles){
        auto it=obstacle.blocks.begin();
        while(it!=obstacle.blocks.end()){
            if(CheckCollisionRecs(it->getrec(),laser.getrec())){
                it=obstacle.blocks.erase(it);
                laser.active=false;
            }
            else{
                ++it;
            }
        }
    }
}

//Alien collision with obstacle:
for(auto& alien:aliens){
    for(auto& obstacle:obstacles){
        auto it=obstacle.blocks.begin();
        while(it!=obstacle.blocks.end()){
            if(CheckCollisionRecs(it->getrec(),alien.getrect())){
                it=obstacle.blocks.erase(it);
            }
            else{
                it++;
            }
        }
    }
    if(CheckCollisionRecs(alien.getrect(),s.getrec())){
        GameOver();
    }
}

}

std::vector<obstacle> game::createobstacles() {

    int obstaclewidth=obstacle::grid[0].size()*3;
    float gap=(GetScreenWidth()-(4*obstaclewidth))/5;
    for(int i=0;i<4;i++){
        float offsetx=(i+1)*gap+i*obstaclewidth;
        obstacles.push_back(obstacle({offsetx,float(GetScreenHeight()-200)}));
    }
    return obstacles;
}

```

```

std::vector<alien> game::createaliens() {

    std::vector<alien>aliens;
    for(int row=0;row<5;row++){
        for(int column=0;column<11;column++){
            int alientype;
            if(row==0){
                alientype=3;

            }
            else if(row==1||row==2){
                alientype=2;
            }
            else{
                alientype=1;
            }
            float x=75+column*55;
            float y=110+row*55;
            aliens.push_back(alien(alientype, {x,y}));
        }

    }
    return aliens;
}

void game::movealiens() {
    for(auto& alien:aliens){
        if(alien.position.x+alien.alienimages[alien.type-1].width>GetScreenWidth()-25){
            aliensdirection=-1;
            movedownaliens(4);
        }
        if(alien.position.x<25){
            aliensdirection=1;
            movedownaliens(4);
        }
        alien.update(aliensdirection);
    }
}

void game::alienshootlaser() {
    double currenttime=GetTime();
    if(currenttime-
lasttimealienfired>=alienlasershootinterval&&!aliens.empty()){
        int randomindex=GetRandomValue(0,aliens.size()-1);
        alien& a=aliens[randomindex];
    }
}

```

```

        alienlaser.push_back(laser({a.position.x+a.alienimages[a.type-
1].width/2,a.position.y+a.alienimages[a.type-1].height},6));
        lasttimealienfired=GetTime();
    }
}

void game::GameOver() {
    run=false;
}

void game::checkforhighscore() {
    if(score>highscore){
        highscore=score;
        savehighscoretofile(highscore);
    }
}

void game::savehighscoretofile(int highscore) {
    std::ofstream highscorefile("highscore.txt");
    if(highscorefile.is_open()){
        highscorefile<<highscore;
        highscorefile.close();
    }
    else{
        std::cerr<<"Failed to save highscore to file"<<std::endl;
    }
}

int game::loadhighscorefromfile() {
    int loadedhighscore=0;
    std::ifstream highscorefile("highscore.txt");
    if(highscorefile.is_open()){
        highscorefile>>loadedhighscore;
        highscorefile.close();
    }
    else{
        std::cerr<<"failed to load from high score file"<<std::endl;
    }
    return loadedhighscore;
}

```

MAIN FUNCTION:

```

#include<string>
#include <raylib.h>
#include"game.hpp"

```

```

#include "laser.hpp"

std::string formatwithleadingzeros(int number,int width){
    std::string numbertext=std::to_string(number);
    int leadingzeros=width-numbertext.length();
    return numbertext=std::string(leadingzeros,'0')+numbertext;
}

int main()
{
    Color grey={29,29,27,255};
    Color blue={0,0,255,255};

    int offset=50;
    int windowheight=700;
    int windowwidth=750;
    InitWindow(windowwidth+offset,windowheight+2*offset,"SPACE SHOOTER");
    InitAudioDevice();
    Font font=LoadFontEx("font/monogram.ttf",64,0,0);
    Texture2D spaceshipimage=LoadTexture("graphics/a.png");
    SetTargetFPS(60);
    game g;

    while(WindowShouldClose() == false) {
        UpdateMusicStream(g.music);
        g.handleinput();
        g.update();

        BeginDrawing();
        ClearBackground(grey);
        DrawRectangleRoundedLines({10,10,780,780},0.18f,20,2,blue);
        DrawLineEx({25,730},{775,730},3,blue);
        if(g.run) {
            DrawTextEx(font,"LEVEL 01",{570,740},34,2,blue);
        } else {
            DrawTextEx(font,"GAMEOVER",{570,740},34,2,blue);
        }
        float x=50.0;
        for(int i=0;i<g.lives;i++){
            DrawTextureV(spaceshipimage,{x,745},WHITE);
            x+=50;
        }

        DrawTextEx(font,"SCORE",{50,15},34,2,blue);
        std::string scoretext=formatwithleadingzeros(g.score,5);
    }
}

```

```

    DrawTextEx(font, scoretext.c_str(), {50, 40}, 34, 2, blue);
    DrawTextEx(font, "HIGH-SCORE", {570, 15}, 34, 2, blue);
    std::string highscoretext = formatwithleadingzeros(g.highscore, 5);
    DrawTextEx(font, highscoretext.c_str(), {655, 40}, 34, 2, blue);

    g.draw();
    EndDrawing();

}

CloseWindow();
CloseAudioDevice();
UnloadFont(font);
UnloadTexture(spaceshipimage);

return 0;
}

```

HOW TO SETUP THIS PROJECT?

1. Visit the website <https://www.raylib.com/>.
2. Click on Download Button.
3. New window pop-up, then click on the link [No thanks, just take me to the downloads](#) on the window.
4. Click on Download.
5. Run the downloaded executable file.
6. Install Raylib.
7. Unzip the provided folder.
8. Click on the main (workspace file).
9. Press F5 to run the game.